

IA Aplicada com LLMs

Aula 07 — As formas de recuperar

A decisão que ficou implícita

Já existe texto — a nota anterior o tirou do PDF. Faltava a outra decisão de projeto: **qual busca**.

RAG abrevia *retrieval-augmented generation*. Nada no nome nem na arquitetura diz que a recuperação é **vetorial**.

O aluno chega aqui supondo que *recuperar* significa *embutir e comparar*. Essa confusão custa caro: indexa-se por vetor o que deveria ser consulta, e depois se ajusta *chunking* para consertar o que nunca foi de *chunking*.

A pergunta que classifica

Antes da ferramenta, o **formato da resposta**:

A resposta é um...	Exemplo	Recuperação
registro	<i>"quem aprovou a D-4471?"</i>	consulta estruturada
termo	<i>"onde aparece 'força maior'?"</i>	busca textual
assunto	<i>"posso reembolsar almoço?"</i>	busca vetorial
relação	<i>"que artigos alteram o teto?"</i>	grafo
agregação	<i>"quais os temas recorrentes?"</i>	grafo, ou pré-cálculo

Só a terceira linha pede *embedding*. É a frase que esta nota inteira sustenta.

Consulta estruturada — extrair parâmetro e consultar

"quem aprovou a despesa D-4471?"

regex `r"D-\d{4}"`
zero chamadas
quebra se a
redação variar

LLM com schema
uma chamada
generaliza a
redação

SELECT aprovador FROM despesas WHERE id = 'D-4471'

Isto já é *tool calling*: a descrição é o prompt, o schema é o contrato.

Dois avisos sobre a consulta estruturada

Text-to-SQL é a versão arriscada. O modelo pode consultar tabela que não devia, gerar varredura completa, ou compor a consulta com texto do usuário.

A forma defensável: o modelo **preenche parâmetros** de uma consulta escrita pela aplicação — o *allowlist* da Aula 05, aplicado a banco.

E o critério que decide antes de tudo:

O teste de um minuto: se um `SELECT` responde a pergunta, não há RAG a construir.

Busca textual — o índice invertido

índice vetorial	chunk —> vetor	"o que se parece com isto?"
índice invertido	termo —> [documentos]	"onde aparece esta palavra?"

BM25 em uma frase: pontua alto o termo que aparece muito **neste documento** e pouco **no corpus** — raridade vale mais que frequência.

Ganha do vetor em	Por quê
identificador e código	D-4471 é literal, não é assunto
nome próprio raro	o vetor não o distingue de um parecido
citação exata	quem pergunta quer aquela frase

E já está instalada: tsvector + GIN no PostgreSQL, FTS5 no SQLite.

A busca vetorial, e as cegueiras relidas

A única que tolera sinônimo e paráfrase. E a única **probabilística**.

O que ela erra é a nota 02 desta aula inteira — identificador, negação, magnitude, tempo:

Três das quatro são casos que um **WHERE** resolve de forma exata e barata.

A quarta é caso de busca textual.

Não são defeito do RAG: são sintoma de ter escolhido **uma** das formas para um corpus que precisava de duas.

O grafo

A pergunta que as outras formas não alcançam

O problema: a resposta não é um objeto

"Uma refeição de R\$ 340,00 por pessoa em Lisboa está dentro da política?"

Exige **três** artigos, e eles estão **ligados entre si**:

Art. 4º §1º	fixa o teto da categoria — R\$ 120,00
Art. 4º §2º	excetua esse teto em viagem internacional — R\$ 260,00
Art. 9º §2º	define quem aprova a faixa resultante

Nenhum dos três responde sozinho. **A relação entre eles responde** — e a relação não está em nenhum dos textos: está *entre* eles.

O vetor mede parecença de assunto, e **"excetua" não é um assunto**.

Nó, aresta, propriedade

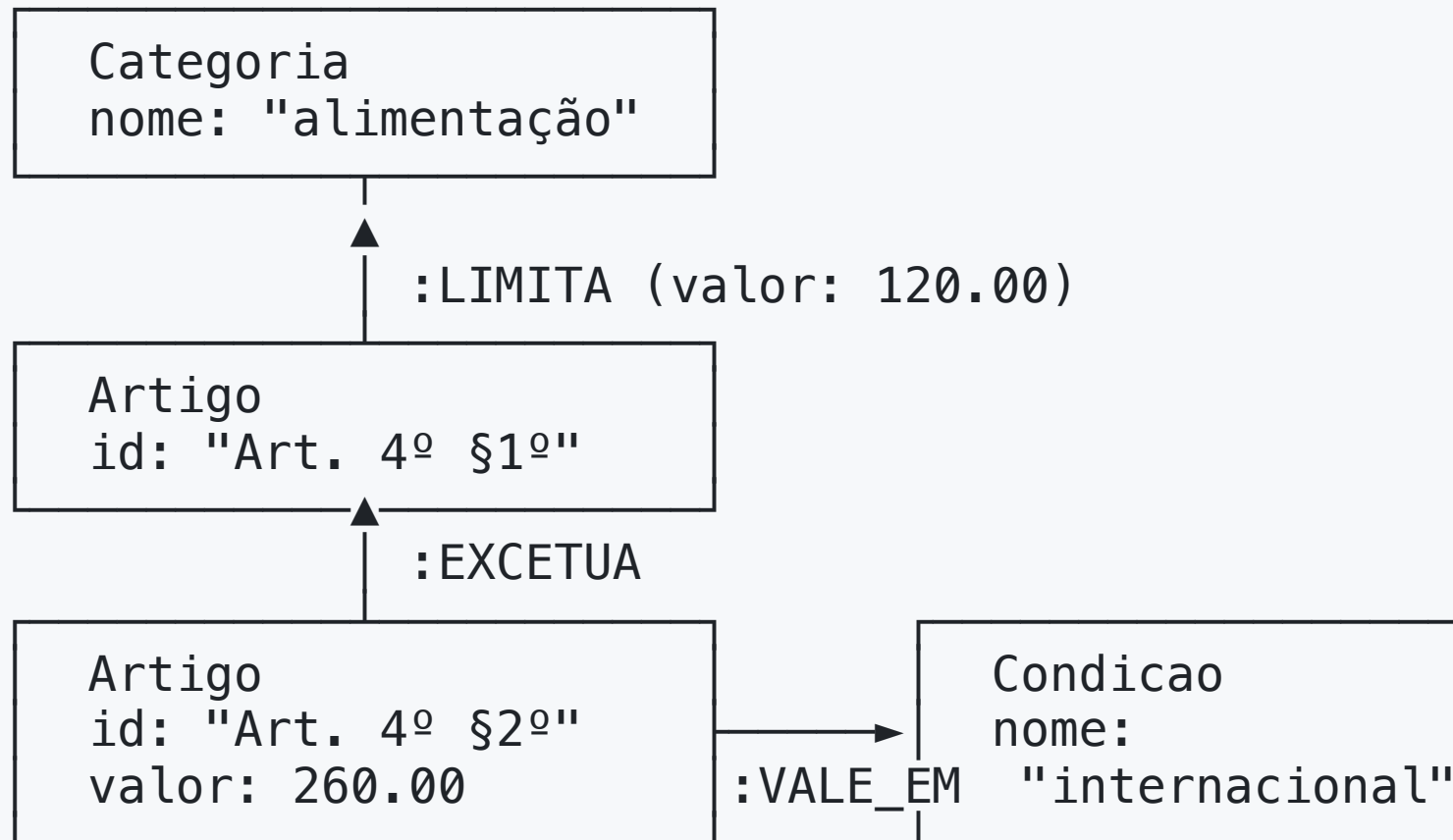
Um banco de grafo — o **Neo4j** é o de uso mais corrente — guarda três coisas:

	O que é	Exemplo
nó	a entidade	um artigo, uma categoria, uma faixa
aresta	a relação, dirigida e tipada	EXCETUA , LIMITA , APROVA
propriedade	o dado no nó ou na aresta	valor: 120.00

A diferença para a tabela é onde a relação mora. Numa tabela, o vínculo é uma chave estrangeira que só existe quando alguém escreve o `JOIN`.

No grafo, a aresta é **objeto de primeira classe, guardada junto do nó**:
percorrer é um salto de ponteiro, e o custo não cresce com o tamanho do banco.

O grafo do regulamento



O Art. 4º §2º não é um artigo parecido com o §1º — é um artigo que

aponta para ele, com um tipo de relação que diz o que faz com ele

Cypher — o padrão é o desenho

A linguagem do Neo4j. `(a)-[:REL]->(b)` é um nó, uma seta e outro nó.

```
CREATE (a41:Artigo {id: 'Art. 4º §1º', valor: 120.00})
CREATE (a42:Artigo {id: 'Art. 4º §2º', valor: 260.00})
CREATE (alim:Categoria {nome: 'alimentação'})
CREATE (intl:Condicao {nome: 'internacional'})

CREATE (a41)-[:LIMITA {valor: 120.00}]->(alim)
CREATE (a42)-[:EXCETUA]->(a41)
CREATE (a42)-[:VALE_EM]->(intl)
```

Ler a consulta é ler o desenho: **procura-se um padrão no grafo**, e o banco devolve todas as ocorrências dele.

A consulta de Lisboa

```
MATCH (base:Artigo)-[:LIMITA]->(:Categoria {nome: 'alimentação'})
OPTIONAL MATCH (excecao:Artigo)-[:EXCETUA]->(base)
WHERE (excecao)-[:VALE_EM]->(:Condicao {nome: 'internacional'})
MATCH (alcada:Artigo)-[:APROVA]->(f:Faixa)
WHERE f.min <= 340.00 <= f.max
RETURN base.id, excecao.id, excecao.valor, alcada.id, f.quem
```

base.id	excecao.id	excecao.valor	alcada.id	f.quem
"Art. 4º §1º"	"Art. 4º §2º"	260.00	"Art. 9º §2º"	"gestor da área"

Uma consulta devolve os três artigos que o pipeline vetorial não reuniu.

O que só o grafo tem: caminho de comprimento variável

```
MATCH caminho = (a:Artigo {id: 'Art. 4º §1º'})  
                <-[:EXCETUA|REMETE_A*1..5]-(afetado:Artigo)  
RETURN [n IN nodes(caminho) | n.id] AS cadeia
```

"Que artigos, direta ou indiretamente, alteram o limite de refeição?"

O `*1..5` percorre a cadeia **sem saber de antemão quantos saltos existem**.

Em SQL, isso é uma CTE recursiva. No vetor, **não é nada**.

E a agregação

```
MATCH (a:Artigo)-[:EXCETUA]->(base:Artigo)-[:LIMITA]->(c:Categoria)
RETURN c.nome, count(a) AS excecoes
ORDER BY excecoes DESC
```

c.nome	excecoes
"alimentação"	1
"hospedagem"	1
"transporte"	0

A resposta **não está em nenhum trecho do documento**. Ela só existe depois de contar — e é por isso que a linha "agregação" não aponta para busca nenhuma.

O preço: está na construção, não na consulta

Custo	O que é
extrair entidades e relações	alguém decide que <code>EXCETUA</code> existe e onde ocorre — manual é caro, automático com LLM é o <i>GraphRAG</i> , e erra
manter	o regulamento muda e o grafo muda junto; um índice vetorial se reconstrói sozinho, um esquema não
operar	mais um serviço, mais um modelo de dados, mais uma linguagem

O meio-termo que costuma bastar: para cadeia rasa e conhecida, um campo `excetua_id` e um `JOIN` resolvem. O grafo se paga quando a profundidade é desconhecida de antemão — que é o que o `*1..5` expressa e o `JOIN` não.

Híbrida — e por que somar escores não funciona

O problema	cosseno vive entre -1 e 1 ; BM25 não tem teto
Somar é	comparar régua com balança
A saída	fundir por posição : $1/(k + \text{posição})$ em cada lista
O nome	<i>reciprocal rank fusion</i> — não precisa de calibragem
O filtro por campo	entra antes da busca, não depois

O preço: dois índices a sincronizar, duas latências, um parâmetro a calibrar.

A escada

	<u>CUSTO</u>	<u>QUANDO É O DEGRAU CERTO</u>
regex	zero	o campo tem formato fixo
consulta estruturada	zero ou uma	a resposta é um REGISTRO
busca textual	zero	termo exato, citação, id
busca vetorial	uma (embedding)	a resposta é uma PASSAGEM
grafo	construção cara	a resposta é o CAMINHO, ou a AGREGAÇÃO

A cascata do *router* da Aula 05, num domínio novo: **começa-se no degrau mais barato que resolve.**

O que fica

- Recuperar tem **cinco formas**, e "RAG" não nomeia nenhuma delas
- Só "**assunto**" pede **embedding** — registro, termo, relação e agregação têm outra resposta
- As cegueiras do vetor são, em três dos quatro casos, um **WHERE**
- O **grafo** responde a pergunta de **caminho** — e o preço está na construção
- Começa-se no **degrau mais barato** que resolve

Próxima nota: o que o vetor **não** representa — as quatro cegueiras, por extenso.